

AIQT CLI

Milestone 3 Build Specification v0.3

Work Graph Planning Foundation

Build-planning document for implementing AIQT Milestone 3. Not a runtime document model for AIQT user projects.

Field	Value
Product	AIQT CLI
Specification	Milestone 3 Build Specification
Version	v0.3
Milestone	M3 - Work Graph Planning Foundation
Aligns With	AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3
Depends On	M1 - Canonical State Foundation, tag m1-canonical-state-foundation M2 - Project Context Capture, merge commit 67d7869, tag m2-project-context-capture
Primary Agent Target	Claude Code or equivalent coding agent
Implementation Status	Draft for final review before build handoff

Document status: Ready for technical review before build handoff

Contents

- 1. Source Alignment
- 2. v0.3 Change Summary
- 3. Milestone Objective
- 4. Planning Model Decision
- 5. Commands in Scope
- 6. Out of Scope
- 7. Behavioral Position in the Workflow
- 8. Plan Input Modes
- 9. Plan Input Contract
- 10. Canonical Work Graph Mutation Rules
- 11. ID Generation and ClientKey Resolution
- 12. Work Unit and Milestone Status Rules
- 13. Dependency Rules
- 14. Command Behavior
- 15. Runlog Rules
- 16. State Outcomes
- 17. Repository Implementation Scope
- 18. Data and Schema Contracts
- 19. Error Handling and Exit Codes
- 20. Testing Scope
- 21. Implementation Tasks
- 22. Definition of Done
- 23. Agent Execution Packet
- Appendix A - Minimum Test Matrix
- Appendix B - Example Plan Input File

1. Source Alignment

This document is derived from AIQT MVP Specification v0.6 and AIQT Technical Architecture Specification v0.3. It assumes Milestone 1 and Milestone 2 are complete on main, with M2 merged at commit 67d7869 and tagged as m2-project-context-capture.

The repository is expected to contain the canonical state foundation, aigt update, planningContextReady helper, strict update schemas, project update service, runlog store extensions, deterministic output renderers, and the full M1/M2 test suite.

The governing design remains:

state -> graph -> packet -> checkpoint

not:

spec -> milestone file -> task file -> review file

Milestone 3 adds the first real work graph capability. It stores implementation milestones, work units, and dependencies as structured records in state.json. It must not create milestone markdown files, task markdown files, agent packets, checkpoints, review reports, or exports.

2. v0.3 Change Summary

- Updates the M2 dependency from future/in-progress to complete: main merge commit 67d7869 and tag m2-project-context-capture.
- Makes structured plan ingestion the only planning model for M3. No autonomous AI planning is implemented.
- Removes ambiguity around orphan work units: every M3 work unit must reference a milestoneClientKey.
- Defines exact status assignment for milestones and work units.
- Defines currentMilestoneId/currentWorkUnitId behavior after plan ingestion.
- Defines non-empty graph behavior: M3 blocks rather than replans.
- Defines dependency direction, duplicate detection, and circular blocking dependency rejection.
- Defines no-input, from-file, JSON, and example modes with deterministic exit codes.
- Adds explicit no-mutation guarantees for failed plan attempts.
- Tightens existing non-empty work graph behavior: state remains unchanged and nextRecommendedCommand remains unchanged.
- Clarifies aigt plan gate order so the no-input gate is evaluated before planning readiness.
- Clarifies that projectStatus = blocked is reserved for future external/manual gates and is not produced by M3.
- Clarifies that circular blocking dependencies are invalid plan input and return exit code 3.

3. Milestone Objective

Milestone 3 implements aigt plan as the deterministic work graph planning foundation. The milestone lets a user or external AI assistant provide a structured plan file, then AIQT validates and stores that plan as the canonical work graph in .aiqt/state.json.

Milestone 3 must prove that AIQT can:

- read the project context captured by M2;
- verify that project context is ready for planning using the M2 planningContextReady helper;
- ingest a structured work graph from a JSON file;

- create milestone records in state.json;
- create work-unit records in state.json;
- create dependency records in state.json;
- assign stable canonical IDs in deterministic input order;
- validate dependency references, duplicate dependencies, and circular blocking dependencies;
- validate work-unit readiness requirements;
- set currentMilestoneId to the first ready work unit milestone;
- keep currentWorkUnitId null until M4 implements full aiqt next;
- update projectStatus to planned when a valid graph has at least one ready work unit;
- append a work_graph.generated runlog event;
- return deterministic human-readable and JSON output.

Milestone 3 is not an autonomous AI planner. AIQT does not call Claude, OpenAI, GitHub Copilot, or another model in this milestone.

4. Planning Model Decision

M3 uses structured plan ingestion rather than autonomous plan generation.

Reason: deterministic ingestion keeps M3 implementable and testable without pretending the local CLI has a built-in planning intelligence layer. The human project owner may ask an external AI assistant to draft the plan JSON, but AIQT only validates and stores the structured plan.

5. Commands in Scope

```
aiqt plan
aiqt plan --json
aiqt plan --from-file <path>
aiqt plan --from-file <path> --json
aiqt plan --example
```

aiqt plan --example prints a sample plan input JSON to stdout and exits with code 0. It does not read .aiqt/, write state, append runlog events, or create export files.

6. Out of Scope

- autonomous AI planning;
- Claude Code API integration;
- OpenAI API integration;
- aiqt next full agent packet generation;
- agent execution packets;
- currentWorkUnitId mutation to in_progress;
- aiqt checkpoint;
- checkpoint capture;
- aiqt review;
- review engine;
- aiqt export;
- markdown documentation generation;

- AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, or .tasks/;
- replanning, replacing, merging, or editing an existing non-empty work graph;
- automatic source-code modification of user projects;
- GitHub integration.

7. Behavioral Position in the Workflow

M1: init/status/next -> M2: update -> M3: plan -> M4: next agent packet -> M5: checkpoint -> M6: review/export

After M2, project.json contains durable project context, state.projectStatus remains draft, and state.nextRecommendedCommand becomes aigt plan when planningContextReady is true.

After M3, state.workGraph contains milestones, workUnits, and dependencies. A valid M3 acyclic graph must produce at least one ready work unit, so state.projectStatus becomes planned and state.nextRecommendedCommand becomes aigt next. projectStatus = blocked is reserved for future external/manual gates and must not be produced by M3.

M3 prepares the state needed by M4. M4 will implement full agent handoff packet generation through aigt next.

8. Plan Input Modes

8.1 File mode

```
aiqt plan --from-file plan.json
aiqt plan --from-file plan.json --json
```

File mode is the primary M3 mode. AIQT loads a structured plan file, validates it, converts client keys to canonical IDs, writes state.json, and appends a work_graph.generated runlog event.

8.2 No-input mode

```
aiqt plan
```

When no --from-file argument is supplied, aigt plan must not invent a work graph. It returns status = needs_input, requiresHumanInput = true, exitCode = 10, and a suggested action explaining that a structured plan file is required.

8.3 Example mode

```
aiqt plan --example
```

Example mode prints a sample plan input JSON and exits with code 0. It does not require an initialized .aiqt/ project and must not mutate files. If --json is also supplied with --example, the command should reject the combination with exit code 3 to avoid mixing example text with CommandResult JSON.

9. Plan Input Contract

M3 accepts JSON only. YAML, markdown, natural language plans, and folder-based milestone files are out of scope.

```
{
  "milestones": [
    {
      "clientKey": "foundation",
```

```

    "title": "Foundation",
    "objective": "Establish the base application structure."
  }
],
"workUnits": [
  {
    "clientKey": "initialize-app",
    "milestoneClientKey": "foundation",
    "title": "Initialize application structure",
    "objective": "Create the first runnable application shell.",
    "scope": ["Create the application scaffold"],
    "outOfScope": ["Do not implement authentication"],
    "acceptanceCriteria": ["Application starts successfully"],
    "agentContextRefs": [],
    "suggestedFiles": ["package.json", "src/"],
    "validationCommands": ["pnpm test", "pnpm build"]
  }
],
"dependencies": []
}

```

9.1 Required top-level fields

Field	Requirement
milestones	Required array with at least one milestone.
workUnits	Required array with at least one work unit.
dependencies	Optional array. Defaults to [] when omitted.

9.2 Required milestone fields

Field	Requirement
clientKey	Required. Unique within milestones. Used only during ingestion.
title	Required non-empty string.
objective	Required non-empty string.

9.3 Required work-unit fields

Field	Requirement
clientKey	Required. Unique within workUnits. Used only during ingestion.
milestoneClientKey	Required. Must reference an existing milestone clientKey. M3 does not create orphan work units.
title	Required non-empty string.
objective	Required non-empty string.
scope	Required array with at least one non-empty item.
outOfScope	Required array with at least one non-empty item.
acceptanceCriteria	Required array with at least one non-empty item.
validationCommands	Required array with at least one non-empty item.

9.4 Optional work-unit fields

Field	Default
agentContextRefs	Defaults to [].
suggestedFiles	Defaults to [].

9.5 Dependency fields

Field	Requirement
fromClientKey	Required. References the prerequisite/blocker work unit.
toClientKey	Required. References the dependent/blocked work unit.
type	Required. One of blocks, requires, relates_to.
reason	Optional nullable string. Defaults to null.

10. Canonical Work Graph Mutation Rules

- aiqt plan mutates state.json and runlog.jsonl only.
- aiqt plan must not mutate project.json.
- state.workGraph.milestones is written from the supplied plan only when the current graph is empty.
- state.workGraph.workUnits is written from the supplied plan only when the current graph is empty.
- state.workGraph.dependencies is written from the supplied plan only when the current graph is empty.
- M3 does not support replanning an existing non-empty graph.
- If state.workGraph is non-empty, aiqt plan --from-file must block with exitCode = 2, no mutation, and recommend a future replan workflow.
- state.currentMilestoneId is set to the milestone of the selected first ready work unit, if any.
- state.currentWorkUnitId remains null in M3. M4 aiqt next will select and start the current work unit.
- state.lastAgentPacket remains null in M3.
- state.lastUpdatedAt is updated only on successful state mutation.
- state.nextRecommendedCommand becomes aiqt next when a ready work unit exists; otherwise aiqt review.
- AIQT_SCHEMA_VERSION remains 0.5.0 because work graph schemas already exist in the canonical model.

11. ID Generation and ClientKey Resolution

Plan input uses temporary clientKey values. Canonical state uses generated stable IDs.

Record type	Canonical ID prefix	Examples
Milestone	M	M001, M002
Work Unit	WU	WU001, WU002
Dependency	DEP-	DEP-001, DEP-002
Runlog Event	EVT-	EVT-004, EVT-005

Resolution rules:

1. All milestone clientKey values must be unique within the plan file.
2. All workUnit clientKey values must be unique within the plan file.
3. Every workUnit.milestoneClientKey must reference an existing milestone clientKey.
4. dependency.fromClientKey and dependency.toClientKey must reference existing workUnit clientKeys.
5. Canonical IDs are assigned in input order.
6. Milestone.workUnitIds are populated using canonical work unit IDs in work unit input order.
7. WorkUnit.dependencies contains canonical dependency IDs where the work unit is the dependency target.
8. IDs must not be regenerated after state is written.
9. If validation fails at any point, no state mutation occurs.

12. Work Unit and Milestone Status Rules

12.1 Work unit readiness gate

A work unit can be marked ready only if it has:

10. non-empty title;
11. non-empty objective;
12. at least one scope item;
13. at least one outOfScope item;
14. at least one acceptance criterion;
15. validationCommands with at least one item;
16. no unsatisfied incoming blocking dependency.

Because M3 creates a fresh graph and no work unit is done yet, any incoming dependency of type blocks or requires is unsatisfied and keeps the target work unit planned. Dependencies of type relates_to do not block readiness.

12.2 Work unit status assignment

Condition	Status
Valid work unit with no incoming blocks/requires dependency	ready
Valid work unit with one or more incoming blocks/requires dependencies	planned

M3 must not create work units with status in_progress, needs_review, done, replanned, or cancelled.

Because M3 rejects circular blocking dependency graphs, any finite valid graph containing blocks/requires dependencies must have at least one source work unit with no incoming blocking dependency. M3 therefore requires a valid ingested graph to produce at least one ready work unit. A zero-ready-work-unit outcome is not produced by M3 and is reserved for future workflow gates outside plan ingestion.

12.2.1 Valid graph readiness invariant

12.3 Milestone status assignment

Condition	Milestone status
At least one child work unit is ready	ready
No child work unit is ready and at least one child work unit is planned	planned

Every milestone must have at least one referenced work unit. Empty milestones are invalid in M3.

12.4 Next ready work unit selection

The selected next ready work unit is the first ready work unit by workUnits input order after dependency evaluation. state.currentMilestoneId is set to that work unit milestoneId. state.currentWorkUnitId remains null.

13. Dependency Rules

Dependency direction must be explicit and deterministic.

```
{
  "fromClientKey": "setup-project",
  "toClientKey": "build-feature",
  "type": "blocks",
  "reason": "The project must exist before the feature can be built."
}
```

- fromClientKey is the prerequisite/blocker work unit.
- toClientKey is the dependent/blocked work unit.
- type must be one of blocks, requires, relates_to.
- blocks and requires prevent the target work unit from being ready until the source is done.
- relates_to is informational and does not block readiness.
- self-dependencies are invalid.
- duplicate dependencies are invalid. A duplicate is the same fromClientKey, toClientKey, and type.
- circular dependencies are invalid input for the blocking dependency graph formed by blocks and requires and must fail with exit code 3 with no mutation.
- relates_to dependencies are ignored for cycle detection.

14. Command Behavior

14.0 aiqt plan gate order

1. If --example is supplied, handle example mode before reading .aiqt/; example mode never mutates files.
2. Locate .aiqt/ and validate project.json, state.json, schema versions, and runlog health for normal plan execution.
3. If --from-file is missing and --example is not supplied, return exit code 10 before evaluating planningContextReady.
4. Evaluate planningContextReady only after a structured plan input path exists.
5. Verify state.workGraph is empty before accepting a new graph.
6. Load and validate PlanInputSchema, clientKey uniqueness, milestone references, dependency references, duplicates, self-dependencies, and circular blocking dependencies.
7. Apply mutation only after all gates and validation rules pass.

14.1 aiqt plan without input

Purpose: prevent accidental graph invention and tell the user how to provide a structured plan.

Result field	Value
status	needs_input
exitCode	10
requiresHumanInput	true
nextRecommendedCommand	aiqt plan --from-file <path>
mutation	None

14.2 aiqt plan --example

Purpose: print a sample JSON plan input to stdout.

- Does not require .aiqt/.
- Does not read project.json, state.json, or runlog.jsonl.
- Does not mutate files.
- Returns exit code 0.
- If combined with --json, reject with exit code 3.

14.3 aiqt plan --from-file

17. Locate .aiqt/.
18. Read and validate project.json.
19. Read and validate state.json.
20. Inspect runlog health using existing behavior.
21. Verify planningContextReady using the M2 helper.
22. Verify state.workGraph is empty.
23. Read plan input JSON.
24. Strictly validate PlanInputSchema.
25. Validate clientKey uniqueness and references.
26. Validate empty milestone rule.
27. Validate dependency duplicates and cycles.
28. Resolve clientKeys to canonical IDs.
29. Create milestone, workUnit, and dependency records.
30. Assign work unit and milestone statuses.
31. Select first ready work unit if one exists.
32. Update state.projectStatus.
33. Update state.currentMilestoneId if a ready work unit exists.
34. Keep state.currentWorkUnitId null.
35. Set state.nextRecommendedCommand.
36. Append work_graph.generated to runlog.jsonl.
37. Return deterministic human or JSON output.

14.4 JSON output example

```
{
  "status": "passed",
  "action": "plan",
  "projectStatus": "planned",
  "currentMilestoneId": "M001",
  "currentWorkUnitId": null,
  "summary": "Work graph generated.",
  "completedActions": ["Read project.json", "Read state.json", "Generated work graph"],
  "changedFiles": [".aiqt/state.json", ".aiqt/runlog.jsonl"],
  "affectedItems": ["M001", "WU001"],
  "blockingIssues": [],
  "warnings": [],
  "requiresHumanInput": false,
  "nextRecommendedCommand": "aiqt next",
  "data": {
    "milestoneCount": 1,
    "workUnitCount": 1,
    "dependencyCount": 0,
    "readyWorkUnitId": "WU001",
    "readyWorkUnitCount": 1,

```

```

    "plannedWorkUnitCount": 0,
    "workGraphWasEmpty": true
  },
  "exitCode": 0
}

```

15. Runlog Rules

A successful plan creation appends one `work_graph.generated` event.

```

{
  "id": "EVT-004",
  "type": "work_graph.generated",
  "timestamp": "ISO_DATE",
  "actor": "aiqt",
  "summary": "Work graph generated from structured plan input.",
  "relatedIds": ["PROJECT-001", "M001", "WU001"],
  "data": {
    "source": "from-file",
    "milestoneCount": 1,
    "workUnitCount": 1,
    "dependencyCount": 0,
    "readyWorkUnitId": "WU001",
    "nextRecommendedCommand": "aiqt next"
  }
}

```

- Event IDs must continue from the highest existing EVT id.
- `relatedIds` must include PROJECT-001 and every created milestone, work unit, and dependency ID.
- Failed plan attempts must not append runlog events.
- No-input and example mode must not append runlog events.
- Malformed runlog lines continue to behave according to M1/M2 runlog health rules.
- M3 does not emit `work_graph.replanned` because replanning is out of scope.

16. State Outcomes

Condition	projectStatus	currentMilestone Id	currentWorkUnit Id	nextRecommendedCommand	Exit code
Planning context not ready	unchanged	unchanged	unchanged	aiqt update	2
No input supplied	unchanged	unchanged	unchanged	aiqt plan --from-file <path>	10
Valid graph with at least one ready work unit	planned	milestone of first ready work unit	null	aiqt next	0
Reserved future state: all work units unavailable due to external/manual gates (not produced by M3)	blocked (reserved)	null	null	aiqt review (future)	N/A in M3

Existing non-empty work graph	unchanged	unchanged	unchanged	unchanged	2
Invalid plan input	unchanged	unchanged	unchanged	unchanged	3

17. Repository Implementation Scope

Recommended M3 implementation files added or modified:

```

src/
cli/
  commands/
    plan.command.ts
core/
  schema/
    plan-input.schema.ts
  workflow/
    work-graph-readiness.ts
    dependency-graph.ts
services/
  planning-service.ts
state/
  workflow-state-store.ts  # may be extended only if needed
  runlog-store.ts         # may be extended only if needed
  ids.ts                  # may be extended only if needed
workflow/
  planning-readiness.ts    # reuse M2 helper; do not duplicate
  next-action.ts          # may be updated only if needed
cli/
  register-commands.ts
  options.ts

tests/
unit/
  plan-input.schema.test.ts
  planning-service.test.ts
  work-graph-readiness.test.ts
  dependency-graph.test.ts
integration/
  plan.command.test.ts
  plan-next-flow.test.ts
fixtures/
  plans/
    valid-plan.json
    valid-plan-with-dependencies.json
    invalid-plan.json
    circular-dependencies.json
    duplicate-dependencies.json

```

If M2 introduced files under `src/workflow` rather than `src/core/workflow`, M3 must follow the existing repository layout. Do not create duplicate helper locations just because this specification names a recommended path.

18. Data and Schema Contracts

18.1 PlanInputSchema

```
export const PlanInputSchema = z.object({
  milestones: z.array(MilestonePlanInputSchema).min(1),
  workUnits: z.array(WorkUnitPlanInputSchema).min(1),
  dependencies: z.array(DependencyPlanInputSchema).default([]),
}).strict();
```

18.2 MilestonePlanInputSchema

```
export const MilestonePlanInputSchema = z.object({
  clientKey: z.string().min(1),
  title: z.string().min(1),
  objective: z.string().min(1),
}).strict();
```

18.3 WorkUnitPlanInputSchema

```
export const WorkUnitPlanInputSchema = z.object({
  clientKey: z.string().min(1),
  milestoneClientKey: z.string().min(1),
  title: z.string().min(1),
  objective: z.string().min(1),
  scope: z.array(z.string().min(1)).min(1),
  outOfScope: z.array(z.string().min(1)).min(1),
  acceptanceCriteria: z.array(z.string().min(1)).min(1),
  agentContextRefs: z.array(z.string()).default([]),
  suggestedFiles: z.array(z.string()).default([]),
  validationCommands: z.array(z.string().min(1)).min(1),
}).strict();
```

18.4 DependencyPlanInputSchema

```
export const DependencyPlanInputSchema = z.object({
  fromClientKey: z.string().min(1),
  toClientKey: z.string().min(1),
  type: z.enum(["blocks", "requires", "relates_to"]),
  reason: z.string().nullable().optional(),
}).strict();
```

18.5 Canonical record shape notes

- Milestone records must match the existing MilestoneSchema.
- Work unit records must match the existing WorkUnitSchema.
- Dependency records must match the existing DependencySchema.
- All created work units receive createdAt and updatedAt using one timestamp per command execution where practical.
- Milestone records do not require timestamps unless the existing schema already includes them.
- Dependency records do not require timestamps unless the existing schema already includes them.

19. Error Handling and Exit Codes

Case	Exit code	Status	Mutation
.aiqt/ missing	3	failed	No

project.json or state.json invalid	3	failed	No
unsupported future schema version	3	failed	No
planningContextReady false	2	blocked	No
no --from-file and no --example	10	needs_input	No
--example with --json	3	failed	No
--from-file path missing or unreadable	3	failed	No
--from-file invalid JSON	3	failed	No
PlanInputSchema validation fails	3	failed	No
duplicate milestone or workUnit clientKey	3	failed	No
unknown milestoneClientKey	3	failed	No
unknown dependency clientKey reference	3	failed	No
self-dependency	3	failed	No
duplicate dependency	3	failed	No
circular blocking dependency	3	failed	No
empty milestone after reference validation	3	failed	No
existing non-empty work graph	2	blocked	No
successful plan ingestion	0	passed	Yes

All errors must render through the existing CommandResult and Issue structures where possible. JSON mode must not emit terminal formatting.

Gate priority note: for normal aiqt plan execution, the missing-input case is evaluated before planningContextReady. Therefore, aiqt plan without --from-file returns exit code 10 even if project context is not ready. Invalid submitted plan structures, including circular blocking dependencies, return exit code 3. Exit code 2 is reserved for valid workflow-state gates such as planningContextReady=false when a plan file is supplied or an existing non-empty workGraph.

20. Testing Scope

- All existing M1 and M2 tests must continue to pass.
- New unit tests must cover PlanInputSchema, clientKey resolution, ID generation, work-unit readiness, milestone status calculation, dependency validation, and cycle detection.
- New integration tests must cover aiqt plan without input, --example, --from-file, --json, invalid input, existing non-empty graph, and successful state mutation.
- At least one flow test must confirm init -> update -> plan -> status produces a planned project and aiqt next recommendation.
- Tests must confirm project.json is not mutated by aiqt plan.
- Tests must confirm runlog events are appended only on successful plan ingestion.
- Tests must confirm no markdown files are created.
- Tests must confirm invalid inputs leave state.json and runlog.jsonl unchanged.
- Tests must confirm currentWorkUnitId remains null after planning.

21. Implementation Tasks

Task	Scope	Acceptance
M3-T1 - CLI registration	Register plan command and flags.	aiqt plan appears in help; unknown flags fail deterministically.
M3-T2 - PlanInputSchema	Add strict plan input schema.	Valid plan files pass; unknown keys and missing required fields fail exit 3.
M3-T3 - Planning gate	Reuse M2 planningContextReady helper.	Not-ready context blocks with exit code 2 and no mutation.
M3-T4 - ClientKey and ID resolution	Map plan clientKeys to M/WU/DEP IDs.	IDs are deterministic and references resolve correctly.
M3-T5 - Dependency validation	Validate references, self-dependencies, duplicates, and cycles.	Invalid dependencies fail with exit code 3 and no mutation.
M3-T6 - Work graph builder	Create canonical milestones, work units, dependencies.	state.workGraph validates and project.json remains unchanged.
M3-T7 - Status and next selection	Assign planned/ready statuses and select first ready work unit milestone.	state.currentMilestoneId and nextRecommendedCommand are correct.
M3-T8 - Runlog event	Append work_graph.generated on successful mutation.	Event ID continues and data shape is deterministic.
M3-T9 - Output rendering	Human and JSON outputs use CommandResult.	--json key order and data fields are stable.
M3-T10 - Tests and validation	Unit and integration coverage.	pnpm validate passes and M1/M2 behavior is preserved.

22. Definition of Done

- aiqt plan is registered and documented in help output.
- aiqt plan without --from-file does not invent a graph and returns exit code 10.
- aiqt plan --example prints example JSON without mutation.
- aiqt plan --from-file validates structured JSON input.
- Milestones are created as records in state.json, not files.
- Work units are created as records in state.json, not files.
- Dependencies are created as records in state.json, not files.
- Canonical IDs are deterministic and stable.
- Every work unit references a milestone.
- Empty milestones are rejected.
- Work unit readiness is validated.
- Blocking dependencies affect ready/planned status.
- Circular blocking dependencies are rejected.
- state.projectStatus becomes planned when at least one work unit is ready.
- state.projectStatus = blocked is not produced by M3; it is reserved for future external/manual gates, checkpoint/review state, or manual blockers.
- state.currentMilestoneId is set to the milestone of the first ready work unit.
- state.currentWorkUnitId remains null.
- state.nextRecommendedCommand becomes aiqt next when ready work exists.
- work_graph.generated is appended to runlog.jsonl on success.
- Invalid input leaves project.json, state.json, and runlog.jsonl unchanged.
- M3 blocks on an existing non-empty workGraph and does not replan.
- No markdown files are created.

- No agent packets are generated.
- pnpm typecheck, pnpm lint, pnpm test, pnpm build, and pnpm validate all pass.
- M1 and M2 tests still pass.
- The project is ready for Milestone 4: aiqt next agent handoff packet generation.

23. Agent Execution Packet

AGENT EXECUTION PACKET

Role

You are implementing Milestone 3 of AIQT CLI.

AIQT is a local TypeScript CLI workflow state engine for projects built with AI coding agents.

Current Work Unit

ID: AIQT-M3

Title: Work Graph Planning Foundation

Objective: Implement aiqt plan as structured work graph ingestion and validation.

Starting Point

M1 is complete and tagged: m1-canonical-state-foundation.

M2 is complete on main at merge commit 67d7869 and tagged: m2-project-context-capture.

Start from main and create branch: feat/m3-work-graph-planning-foundation.

Scope

Implement:

- aiqt plan
- aiqt plan --json
- aiqt plan --from-file <path>
- aiqt plan --from-file <path> --json
- aiqt plan --example
- structured JSON plan input validation
- milestone record creation in state.json
- work-unit record creation in state.json
- dependency record creation in state.json
- canonical ID generation: M001, WU001, DEP-001
- clientKey to canonical ID resolution
- work-unit readiness validation
- milestone status assignment
- dependency validation and circular blocking dependency rejection
- projectStatus update to planned or blocked
- currentMilestoneId update to first ready work unit milestone
- currentWorkUnitId remains null
- nextRecommendedCommand update to aiqt next when ready
- work_graph.generated runlog event
- deterministic human and JSON output
- tests for the M3 test matrix

Out of Scope

Do not implement:

- autonomous AI planning
- aiqt next agent packet generation
- aiqt checkpoint
- aiqt review
- aiqt export

- replanning existing non-empty graphs
- markdown document generation
- AIQT.md, AGENTS.md, CLAUDE.md
- Claude/OpenAI API calls
- source-code modification of user projects
- GitHub integration

Validation Commands

pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm validate

Required Agent Output

Return:

1. Summary of implementation
2. Files created
3. Files modified
4. Tests added or changed
5. Validation commands run
6. Confirmation that M1 and M2 tests still pass
7. Confirmation that no out-of-scope functionality was implemented
8. Any unresolved issues
9. Suggested next step

Appendix A - Minimum Test Matrix

Test Type	Scenario	Expected Result
Unit	PlanInputSchema validates valid minimal plan	Pass
Unit	PlanInputSchema rejects unknown top-level or nested keys	Exit code 3 through command path
Unit	PlanInputSchema rejects missing milestoneClientKey	Exit code 3 through command path
Unit	Duplicate milestone clientKey	Invalid input, exit code 3
Unit	Duplicate workUnit clientKey	Invalid input, exit code 3
Unit	Unknown milestoneClientKey	Invalid input, exit code 3
Unit	ID generation creates M/WU/DEP IDs	Stable next ID
Unit	Work unit with no incoming blocking dependency	status = ready
Unit	Work unit with incoming blocks dependency	status = planned
Unit	Milestone with ready child work unit	status = ready
Unit	Dependency self-reference	Invalid input, exit code 3
Unit	Duplicate dependency	Invalid input, exit code 3
Unit	Circular blocks/requires dependency	Invalid input, exit code 3
Integration	aiqt plan without input in non-TTY	requiresHumanInput true, exit code 10
Integration	aiqt plan --example	prints JSON example, exit code 0, no mutation
Integration	aiqt plan --from-file valid plan	state.workGraph populated
Integration	aiqt plan --from-file --json	deterministic CommandResult JSON
Integration	planningContextReady false	blocked, exit code 2, no mutation
Integration	existing non-empty workGraph	blocked, exit code 2, no mutation
Integration	successful plan appends work_graph.generated	valid JSONL event

Integration	project.json after plan	unchanged from before plan
Integration	state.currentWorkUnitId after plan	null
Integration	state.currentMilestoneId after plan	first ready work unit milestone ID
Integration	init -> update -> plan -> status	projectStatus = planned; nextRecommendedCommand = aiqt next
Integration	invalid plan input	state.json and runlog unchanged
Integration	no markdown files created	AIQT.md/AGENTS.md/CLAUDE.md/docs absent
Regression	all M1 and M2 tests	Still pass

Appendix B - Example Plan Input File

```
{
  "milestones": [
    {
      "clientKey": "m1-foundation",
      "title": "Application Foundation",
      "objective": "Create the base application structure and development workflow."
    },
    {
      "clientKey": "m2-core-feature",
      "title": "Core Feature",
      "objective": "Implement the first user-facing feature."
    }
  ],
  "workUnits": [
    {
      "clientKey": "wu-init-app",
      "milestoneClientKey": "m1-foundation",
      "title": "Initialize application shell",
      "objective": "Create a runnable app shell with test and build commands.",
      "scope": ["Create package setup", "Create src entry point", "Create test setup"],
      "outOfScope": ["Do not implement authentication", "Do not implement persistence"],
      "acceptanceCriteria": ["pnpm build passes", "pnpm test passes"],
      "agentContextRefs": ["project.objective", "context.technologyPreferences"],
      "suggestedFiles": ["package.json", "src/index.ts", "tests/"],
      "validationCommands": ["pnpm test", "pnpm build"]
    },
    {
      "clientKey": "wu-core-feature",
      "milestoneClientKey": "m2-core-feature",
      "title": "Implement core feature",
      "objective": "Build the smallest useful version of the main feature.",
      "scope": ["Implement core feature logic", "Add tests"],
      "outOfScope": ["Do not add secondary features"],
      "acceptanceCriteria": [
        "Feature works through the expected interface",
        "Tests cover success and failure cases"
      ],
      "agentContextRefs": ["requirements"],
      "suggestedFiles": ["src/", "tests/"],
      "validationCommands": ["pnpm test"]
    }
  ]
}
```

```
],  
"dependencies": [  
  {  
    "fromClientKey": "wu-init-app",  
    "toClientKey": "wu-core-feature",  
    "type": "blocks",  
    "reason": "The application shell must exist before the core feature can be implemented."  
  }  
]  
}
```